Readiculous

AI Agent Workflow & Configuration Summary

Prepared for Saris AI Application Submission

Working with AI Coding Agents

The most valuable rule I have developed while working with AI coding agents is forcing the model to ground itself in source-of-truth artifacts before making changes. In this repository that meant treating schema.sql, retraining contracts, and API payload examples as authoritative instead of allowing the agent to infer interfaces. I added the anti-hallucination section after repeatedly seeing agents invent field names, ports, and join keys that were “almost correct” but operationally dangerous.

I also rely heavily on executable validation loops instead of trusting generated code by inspection. The user_flow_cli.js integration driver became especially valuable because it exercises the full backend + ML pipeline end-to-end without needing the mobile frontend. Over time I’ve found that lightweight executable workflows and explicit interface contracts produce much more reliable AI-assisted development than large abstract prompting rules.

How I Actually Use AI Coding Agents

- Force the agent to read schema.sql and the relevant route file before touching any data layer — no assumptions about field names
- Treat README + schema as grounding context, not background reading; paste relevant sections directly into context for high-stakes edits
- Prefer incremental diffs over large rewrites — smaller surface area means hallucinations are easier to catch
- Use CLI integration flows (user_flow_cli.js pattern) as validation loops instead of trusting generated code by inspection alone
- Ask the agent to summarize its assumptions before implementation, then verify each assumption against source
- Never trust generated API field names without a grep confirmation against the actual route handler
- Keep inter-process contracts (e.g. JSON-to-stdout) explicit and minimal — complexity at boundaries is where agents make the most mistakes
- When the agent proposes a refactor, ask it to identify every callsite first — agents frequently miss indirect usages
- For ML code: pin every numeric constant (thresholds, weights, blend ratios) in comments so the agent cannot silently drift them on a rewrite
- Treat the agent’s first draft as a strong starting point for review, not a finished artifact — the review loop is where the real engineering happens

Executive Summary

This document surveys every AI-agent-relevant configuration, workflow, and architecture file in the Readiculous monorepo. The repository contains **no dedicated agent config files** (no CLAUDE.md, AGENTS.md, .cursorrules, system prompts, or slash commands). Instead it has a set of high-quality engineering artifacts that serve as implicit grounding documents for any AI agent working in this codebase.

The stack is: **Flutter** mobile client → **Node.js/Express** REST backend (MySQL) → **Python/Flask** ML microservice (XGBoost + SVD + Cosine Similarity).

System Architecture

```
Flutter App (iOS + Android)
      |
      | REST API (:5000)
      v
Node.js + MySQL Backend
      |
      | Internal HTTP (:6000)
      v
Python Flask ML Service
  XGBoost + SVD + TF-IDF/Cosine Similarity
```

Key cross-service identifier: **isbn13** is the shared key between the Kaggle CSV, MySQL books table, and ML service. Note: the legacy database/schema.sql names this column isbn — an undocumented discrepancy agents must be aware of.

Files Found and Their Purpose

1. ml/README.md — Primary Architecture + API Reference

Agent submission value: HIGH

The most comprehensive single file in the repo. Covers:

- Full 3-tier architecture diagram
- ML model design: XGBoost (content) + TF-IDF/Cosine (content similarity) + Surprise SVD (collaborative filtering)
- Both recommendation flows: /recommend (reader) and /suggest (library)
- All 40+ REST API endpoints across 9 resource groups
- /compare endpoint for side-by-side model output comparison during testing
- /reload hot-swap endpoint — refreshes .pkl artifacts in memory without process restart
- Cold-start vs. warm-start recommendation routing logic
- Repository structure with all .pkl file locations
- Environment variable requirements and deployment notes

Most important rules for agents:

- Cold-start users (no history) → content-only recommendations
- Partial history → hybrid, weighted toward content
- Rich history → hybrid, weighted toward collaborative filtering
- isbn13 is the cross-system join key (not book_id)
- Required env vars: GOODREADS_CSV, DB_HOST, DB_USER, DB_PASSWORD, DB_NAME
- ML service runs on :6000; backend on :5000

2. ml/notebooks/features/user_book_recommender/retrain.py — ML Retraining Pipeline

Agent submission value: HIGH

The most architecturally significant ML file. Called by the Node.js backend via POST /api/ml/retrain. Outputs a single JSON line to stdout; Node.js parses this as the response.

Pipeline steps:

- Pull quality signals from MySQL (user ratings + library STOCKED/ORDERED/IGNORED decisions)
- Pull user interactions for collaborative filtering (explicit ratings + implicit from status)
- Merge live signals with the Kaggle 100k-book CSV baseline
- Feedback loop: books rated by real users but absent from Kaggle CSV are appended to training data
- Conditionally retrain XGBoost (content-based) and SVD (collaborative filtering)
- Overwrite .pkl files in place; emit structured JSON to stdout

Critical constants agents must not invent:

Constant	Value	Meaning
MIN_NEW_ROWS	100	Skip XGBoost retrain if fewer MySQL signals
MIN_CF_ROWS	10	Skip CF retrain if fewer interactions
Rating blend	0.7 kaggle + 0.3 user_avg	When user rating exists, blend with Kaggle baseline
Library weight: STOCKED/ORDERED	1.0x	Upweight books librarians approved
Library weight: IGNORED	0.5x	Downweight books librarians rejected
Implicit: read	3.5	Finished book — probably liked it
Implicit: reading	3.0	In progress — neutral positive
Implicit: want_to_read	2.5	Expressed interest — weaker signal

Output contract (always one JSON line to stdout):

```
{"status": "ok", "elapsed_s": 12.3, "xgb_status": "ok", "xgb_accuracy": 0.87,
 "cf_status": "ok", "cf_rmse": 0.91, "cf_unique_users": 6, "cf_unique_books": 42}

{"status": "error", "message": "DB_HOST environment variable is not set."}
```

3. backend/scripts/user_flow_cli.js — Interactive E2E Test Driver

Agent submission value: MEDIUM-HIGH

An interactive CLI that exercises the entire backend + ML stack without Flutter. Run with npm run test:user-flow from backend/. This is the closest thing to an integration test suite in the repo.

Covers:

- Reader flow: view reads, add/update book status + rating, generate recommendations, view saved recommendations, trigger ML retrain
- Librarian flow: view library inventory, view local reader activity, generate library recommendations, update recommendation state, trigger ML retrain
- Reveals endpoints not in README: POST /recommendations/users/:id/generate and POST /recommendations/libraries/:id/generate
- Confirms library recommendation state machine: NEW → ORDERED → STOCKED → IGNORED
- Calls POST /api/ml/retrain directly — confirms Node-to-Python subprocess contract

4. backend/DEMO_USERS.md — Seed Credentials

Agent submission value: MEDIUM

Documents deterministic test accounts created by node scripts/seed_demo_data.js. The librarian account is the only account with access to library-role endpoints.

Role	Email	Password
Reader (x6)	e.g. ava.reader@readiculous.demo	AvaReads123!
Librarian	grace.librarian@readiculous.demo	GraceLibrary123!

5. database/schema.sql — Source-of-Truth Data Model

Agent submission value: HIGH

Defines all MySQL tables. Essential anti-hallucination ground truth — agents must cross-reference this before writing SQL or constructing API payloads.

Table	Key Facts
users	role ENUM('user','librarian') — not 'reader'. user_id is VARCHAR(255) UUID.
books	PK is isbn (VARCHAR 20) in schema; ML code calls it isbn13 — discrepancy.
user_reads	Has both status (want_to_read/reading/read) AND rating (0–5 float).
library_recommendations	state ENUM: NEW / ORDERED / STOCKED / IGNORED.
user_genres / book_genres	Many-to-many join tables; genre_id is INT UNSIGNED.
user_libraries	One user → one library (one-to-one PK on user_id).

6. backend/BookRecommendation.postman_collection.json — API Contract Artifact

Agent submission value: MEDIUM

Postman collection with pre-built request bodies for all major endpoints. Useful for extracting exact field names and example payloads without guessing. Backend base URL: http://localhost:5000.

7. .github/workflows/deploy-pages.yml — CI/CD

Agent submission value: LOW

Deploys docs/ to GitHub Pages on push to main. No test or lint enforcement gates. The pipeline is purely a static site deploy.

8. frontend/analysis_options.yaml — Flutter Linter Config

Agent submission value: LOW (as a standalone file)

Enables package:flutter_lints/flutter.yaml with no custom overrides. Confirms flutter analyze is the linting command for the Flutter frontend.

File Summary Table

File	Type	Saris AI Value
ml/README.md	Architecture + full API docs	HIGH
ml/.../retrain.py	ML pipeline with I/O contracts	HIGH
database/schema.sql	Data model ground truth	HIGH
backend/scripts/user_flow_cli.js	E2E test driver (CLI)	MEDIUM-HIGH
backend/DEMO_USERS.md	Seed credentials for demo	MEDIUM
backend/BookRecommendation.postman_collection.json	API payload examples	MEDIUM
.github/workflows/deploy-pages.yml	CI/CD (no test gates)	LOW
frontend/analysis_options.yaml	Flutter linter config	LOW
database/.env.example	Env var contract	LOW

Reusable Patterns for Saris AI

Tiered Cold-Start Routing

content-only → hybrid (content-weighted) → hybrid (CF-weighted) based on interaction count thresholds. A clean, reusable ML routing pattern for any recommendation system.

Library Signal Weighting

Turning domain-expert decisions (librarian STOCKED/ORDERED/IGNORED) into training sample weights (3.0x / 0.5x). A principled human-in-the-loop feedback pattern applicable to any expert-guided ML system.

Implicit Rating Fallbacks

Deriving ratings from behavioral signals when explicit ratings are absent: read=3.5, reading=3.0, want_to_read=2.5. A standard implicit feedback pattern that prevents cold-start failures in collaborative filtering.

JSON-to-stdout Subprocess Contract

Node.js spawns Python; Python always emits exactly one JSON line to stdout (success or error). Simple, robust inter-process contract that avoids stderr/stdout confusion and enables structured error propagation.

CLI-as-Integration-Test

user_flow_cli.js exercises the full backend + ML API without a test framework. A lightweight pattern for validating end-to-end flows during development when a full test suite isn't yet in place.

Feedback Loop via Training Data Expansion

Every book a real user rates (even if absent from the Kaggle baseline) gets appended to the XGBoost training set. The model improves as the user base grows without requiring a full dataset replacement.

Implicit vs. Explicit Operational Context

The repository relies on **implicit operational context embedded across README, schema, CLI tooling, and ML retraining code** rather than centralized agent instructions. This is a deliberate engineering choice — the contracts live close to the code that enforces them rather than in a separate instruction layer that can drift. The practical consequence is that an agent working in this codebase must be explicitly directed to read the right artifacts before acting.

Context Gap	Where the Truth Actually Lives
No centralized agent instruction file (CLAUDE.md / AGENTS.md)	Architecture is in README.md; contracts in retrain.py docstring and user_flow_cli.js
No automated test suite	user_flow_cli.js is the manual integration driver; run it to validate the full stack
No ESLint config for backend JS	No static analysis gate — code review is the only enforcement layer
No OpenAPI/Swagger spec	Endpoint contracts in README tables + Postman collection + route handlers directly
isbn vs isbn13 discrepancy undocumented	schema.sql PK is 'isbn'; ML service normalizes to 'isbn13' at CSV load time in retrain.py:267
Generate endpoints not in README	POST /recommendations/users/:id/generate and /libraries/:id/generate visible only in user_flow_

Anti-Hallucination Safeguards for Agents

The following facts are non-obvious and likely to be hallucinated incorrectly by an agent without this document:

- user role value is 'user' (not 'reader') in the ENUM
- ML service runs on port 6000; backend on 5000 — never swap these
- The Kaggle CSV column is named 'isbn'; the codebase normalizes it to 'isbn13' at load time
- Implicit rating values are exactly 3.5 / 3.0 / 2.5 — not 4 / 3 / 2
- Library signal weights are exactly 3.0 (approved) and 0.5 (rejected)
- Rating blend is 70/30 (kaggle/user), not 50/50
- XGBoost skips retrain below 100 signals; SVD skips below 10 interactions
- The /reload endpoint reloads .pkl files in memory — it does not retrain
- SessionNotifier (Riverpod) is the live auth layer; AuthService is legacy dead code
- user_libraries is a one-to-one table (user_id is PK) — one user can only belong to one library

Generated from live codebase analysis. Verify against current source before asserting any claim as authoritative — the codebase evolves.